

Lecture - 3

If-Else Condition, Operators and Basic of loops

* Why We Write return 0?

return 0 tells that the program ends now. and after return 0 program is returned and terminated.

* Why "using namespace std"?

We can define the properties of keywords using "using namespace std".

→

```
int main() {  
    int Cout = 10;  
    Cout << Cout; } → It gives error because it  
                        can't understand.
```

→

```
int main() {  
    int Cout = 10;  
    std::Cout << Cout; } → It works properly.
```


* Write a program to check given character is Vowel or Consonant.

Vowels : a e i o u
if ~~if~~ character is Vowel
then print Vowel

Otherwise

Print Consonant.

In this we use OR operator.

a OR e OR i OR o OR u

~~#include <stdio.h>~~

#include <iostream>

using namespace std;

int main() {

char c = 'u';

if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u')

cout << "Vowel";

else

cout << "Consonant";

return 0;

}

Output = Vowel

* Write a program to check whether given year is leap or not.

For leap:

$$\text{year \% } 400 = 0$$

↳ then leap year

$$\text{year \% } 4 = 0 \text{ \& year \% } 100 \neq 0$$

↳ then leap year

Otherwise

↳ Not Leap year.

→ #include <iostream>
using namespace std;

int main() {

int year;

cin >> year;

// Take input by user.

if (year \% 400 == 0)

cout << "leap year";

else if (year \% 4 == 0 \&\& year \% 100 != 0)

cout << "leap year";

else

cout << "Not a leap year";

return 0;

}

Output:

1996

leap year

* Given a number, check if it is divisible by 3 & 5?

Ex : 10

$$10 \% 3 = 1 \quad \times$$

Not perfect

$$15 \% 3 = 0 \quad \checkmark$$

$$15 \% 5 = 0 \quad \checkmark$$

perfect

9

$$9 \% 3 = 0 \quad \checkmark$$

$$9 \% 5 = 4 \quad \times$$

Pseudocode :

Not perfect.

```

if (num % 3 == 0) {
    if (num % 5 == 0) {
        cout << "Perfect";
    } else {
        cout << "Not perfect";
    }
}
else {
    cout << "Not perfect";
}

```

Code :

```

#include <iostream>
using namespace std;

```

```

int main() {
    cin >> num;
}

```



```
if (num % 3 == 0 && num % 5 == 0)
```

```
    cout << " perfect";
```

```
else
```

```
    cout << "perfect Not perfect";
```

```
    return 0;
```

```
}
```

→

Output:

1)

15

Perfect

2)

20

Not perfect

*

For taking input From user:

cin >> Variable-name;

cin : character input or Console input

>> → Extraction

* Add two number by taking input from user:

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
int num1, num2;
```

```
cin >> num1;
```

```
cin >> num2;
```

```
int sum;
```

```
sum = num1 + num2;
```

```
Cout << "Sum is " << sum;
```

```
return 0;
```

```
}
```

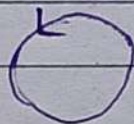
Output

→ 24

→ 26

50

* Loops :



Repeat one task a finite number of time.

```
for (initialize ; condition ; decrement  
operation ) {  
increment
```

ll lines of code

```
}
```

→ We want to print whole number from 1 to 100.

for this we have to write 100 line of code.

```
Cout << 1;
```

```
Cout << 2;
```


From For loop We can write easily.

```
for (int i = 1; i <= 100; i++) {
    cout << i;
}
```

Code for printing 1 to n.

```
#include <iostream>
using namespace std;
```

```
int main() {
    int n;
    cin >> n;
```

```
for (int i = 1; i <= n; i++) {
    cout << i << " ";
}
return 0;
}
```

Output

20

1 2 3 4 5 6 7 8 9
10 11 12 13 14 15 16 17 18
19 20

* Pre increment & post increment :-

↓
++num

Sum = 0

num = 0;

Sum = num++

Sum = 0 + 1

Sum = 1 + 1
= 2

↓
num++

first print then add.

num = num + 1

first add then print.

num = 0

cout << ++num;

↓

Output = 0

cout << num++;

↓

Output = 1.

* print all even number 1 to 20

2, 4, 6, 8, 10, 12, 14, 16, 18, 20

2%2 4%2 6%2

= 0 = 0 = 0

num%2 = 0

↳ then it is odd.

We can solve this by for loop

to check all number present inside limit.

Code :

```
#include <stdio.h>

using namespace std;

int main() {
    for (int i = 0; i <= 20; i++) {
        if (i % 2 == 0)
            cout << i << " ";
    }
    return 0;
}
```

Output :-

2 4 6 8 10 12 14 16 18 20

* code for sum of first n natural number.

let $n = 10$;

start from 1; $i = 1$;

Initially, $Sum = 0$

$$Sum = 0 + 1 = 1$$

$$i = i + 1$$

$$Sum = 1 + 1 = 2 \quad i = 3$$

$$Sum = 5$$

↓

$$Sum = 55$$

Code %

```
#include <stdio.h>
using namespace std;
```

```
int main() {
```

```
    int n;
```

```
    cin >> n;
```

```
    sum = 0;
```

```
    for (int i = 1; i <= n; i++) {
```

```
        sum = sum + i;
```

```
    }
```

```
    cout << sum;
```

```
    return 0;
```

```
}
```


Lecture - 4For loop + While loop + pattern printing

A pandit say "OM" 108 times till
locket does not end.



For repeating one work we use loops:

If Computer is a pandit then he write:

```
for (int i=0; i<108; i++)
    cout << "OM";
```

* Write a table using program:

$$3 * 1 = 3$$

$$3 * 2 = 6$$

⋮

$$3 * 10 = 30$$

Constant

Increment
by 1.

→ $3 \times i$ (always)

i have some limit till where it go.


```
for (int i = 1; i <= 10; i++) {
    cout << "3 * " << i << " = " << 3 * i << endl;
}
```

* Write a program to check prime Number:

Prime number: divided by 1 & itself.

Let check for 15

We can check 2 to 14

if any number from 2 to 14 can divide 15 then it is not prime.

$$15 \% 2 == 1 \times$$

$$15 \% 3 == 0 \checkmark \text{ (Not prime)}$$

Break the loop.

* code:

```
#include <iostream>
using namespace std;
int main() {
```

```
    int num;
```

```
    cin >> num;
```

```
    for (int n = 2; n <= num - 1; n++) {
```

```
        if (num % n == 0) {
```

```
            cout << "Not prime";
```

```
            break;
```

```
        }
```

```
    cout << "prime Number";
```

```
}
```

```
    return 0;
```

```
}
```


break : If we put break, then it exit from loop permanently.

put some edge case :

1 is not prime.

Code :

```
int main() {
    int n;
    cin >> n;
    if (n < 2) {
        cout << "Not prime";
        return 0;
    }
    for (int num = 2; num <= n-1; num++) {
        if (n % num == 0) {
            cout << "Not a prime";
            return 0;
        }
    }
    cout << "prime";
    return 0;
}
```


* Fibonacci Series :-

0 1 1 2 3 5 8 13 21 34

→ 1st element → 0 (fixed)

→ 2nd element → 1 (second)

Third → 1st + 2nd = 0 + 1 = 1

4th → 2nd + 3rd = 1 + 1 = 2

5th → 3rd + 4th = ~~2+2~~ 1 + 2 = 3

$$n^{\text{th}} \rightarrow (n-1)^{\text{th}} + (n-2)^{\text{th}}.$$

First = 0

Second = 1

Current = 0 + 1 = 1 (Third)

Now,

First ← second

First = 1

Second ← current

second = 1

current = 1 + 1 = 2 (Fourth)

→ Code :

```
int n;
```

```
cin >> n;
```

```
int first = 0;
```

```
int second = 1;
```

```
int current;
```

```
for (int i = 3; i <= n; i++) {
```

```
    current = first + second;
```



```

    first = second;
    second = current;
}

```

```

    cout << current;
    return 0;
}

```

→ The upper code work on $n > 2$.

→ Edge case. (put before loop)

```

if (n < 3)
    return (n-1);

```

or

We write

```

if (n == 1 || n == 2)
    return (n-1);

```

*

$n = 3 > 2$; // True

$cout << n$; // output = 1

$n = 3 > 2 > 1$;

$cout << n$;

$n = (3 > 2) > 1$
1

$= 1 > 1$ // False

Output = 0.

* Operator : (precedancy)

Operator

Associativity

(), [], → , .

Left to right

unary !, ~, ++, --, +, -, *, &
(type), sizeof

Right to left

Arithmetic *, /, %
+, -

Left to Right

Shift << >>

”

Comparison <, <=, >, >=

”

==, !=

”

Bitwise &

”

logical &&, ||, !

”

Assignment =, +=, -=, *=, /=

Right to left.

!=, <<=, >>=

$\eta = (3 > 2) > 1$

$1 > 1 \rightarrow \text{false}$

return 0;

* Pattern :

1)

					Col 1	2	3	4
Row 1	1	1	1	1	00	01	02	03
Row 2	1	1	1	1	10	11	12	13
Row 3	1	1	1	1	20	21	22	23
Row 4	1	1	1	1	30	31	32	33

First Approach:

Make 4 for loop and in every for loop print 1 four times.

```
for (int i = 1; i <= 4; i++)
```

```
    cout << "1";
```

```
for (int i = 1; i <= 4; i++)
```

```
    cout << "1";
```

What if pattern is of 100 lines

In this we can repeat same loop.

We have to use loop for this. We use nested loop here.

(loop ke andar loop):-

```
int main() {
```

```
    for (int i = 0; i < 4; i++) {
```

```
        for (int j = 0; j < 4; j++) {
```

```
            cout << "1";
```

```
        }
```

```
        cout << endl;
```

```
    }
```

```
    return 0;
```

```
}
```


2)

1 2 3 4

1 2 3 4

1 2 3 4

1 2 3 4

00 1	01 2	02 3	03 4
10 1	11 2	12 3	13 4
20 1	21 2	22 3	23 4
30 1	31 2	32 3	33 4



We use nested loop.

It print value of
(col + 1) in
each box.

```
int main() {
```

```
    for (int row = 0; row < 4; row++) {
```

```
        for (int col = 0; col < 4; col++) {
```

```
            cout << col + 1;
```

```
        }
```

```
        cout << endl;
```

```
    }
```

```
    return 0;
```

```
}
```

3)

4 3 2 1

4 3 2 1

4 3 2 1

4 3 2 1

00 4 01 3 02 2 03 1

10 4 11 3 12 2 13 1

20 4 21 3 22 2 23 1

30 4 31 3 32 2 33 1

At Every box

(4 - col):

```
for (int row = 0; row < 4; row++) {
```

```
    for (int col = 0; col < 4; col++) {
```

```
        cout << 4 - col;
```

```
    }
```

```
    cout << endl;
```

```
}
```


4)	1	2	3	4	00(1)	01(2)	02(3)	03(4)
	5	6	7	8	10(5)	11(6)	12(7)	13(8)
	9	10	11	12	20(9)	21(10)	22(11)	23(12)
	13	14	15	16	30(13)	31(14)	32(15)	33(16)

At Every box it is incremented by 1
Start With

Count = 1

When it prints:

then Count++;

code :

```

int Count = 1;
for (int row = 0; row < 4; row++) {
    for (int col = 0; col < 4; col++) {
        Count << Count << " ";
        Count++;
    }
    Count << endl;
}

```

5)	*				00
	*	*			10 11
	*	*	*		20 21 22
	*	*	*	*	30 31 32 33

Every row में एक नमूना आता है।

row की value नमूना

1 row = 1

2nd row = 2

3rd row = 3

Code :

```

int n = 4;
for (int row = 0; row < n; row++) {
    for (int col = 0; col <= row; col++) {
        cout << "* ";
    }
    cout << endl;
}

```

5)

1					00 1
1	2				10 1 11 2
1	2	3			20 1 21 2 22 3
1	2	3	4		30 1 31 3 32 3 33 4

It stops when
col <= row

and

prints col + 1.

```

for (int row = 0; row < 4; row++) {
    for (int col = 0; col <= row; col++) {
        cout << col + 1;
    }
}

```

```

    cout << endl;
}

```


6) a a a a 00 a 01 a 02 a 03 a
 b b b b 10 b 11 b 12 b 13 b
 c c c c 20 c 21 c 22 c 23 c
 d d d d 30 d 31 d 32 d 33 d

1st Approach:

Make 4 loop one by one
 and print 'a' in first loop
 and 'b' in second loop
 and increase in Next loop.

```
for (int i = 0; i < 4; i++) {
    cout << "a";
}
for (int i = 0; i < 4; i++) {
    cout << "b";
}
```

2nd Approach (By Nested loop):-

In this it uses typecasting.

In 1st loop it print a

ASCII value of a = 98

Code:

```
for (int row = 0; row < 4; row++) {
    char c = 'a' + row;
    for (int col = 0; col < 4; col++) {
        cout << c << " ";
    }
    cout << endl;
}
```


009

10 a 11 b

20a 21b 22c

30 a 31 b 32 c 33 d

→ In every row print till row number.

→ what to print

→ at every column change the value changes.

```
for (int row = 0; row < 4; row++) {  
    char c = 'a';  
    for (int col = 0; col < 4; col++) {  
        c = 'a' + 0;  
        cout << c << " ";  
    }  
}
```

```
cout << endl;
```

3